

HERO: A Heterogeneous Energy-Aware Runtime Orchestrator for Quantum-Classical Cybersecurity Workloads

Abstract: Near-term quantum computing for cybersecurity will inherently be hybrid, where the classical CPU controls the flow of computation, the classical GPU executes the heavy data processing steps, and the Quantum Processing Unit (QPU) performs the short quantum algorithms. Previous studies benchmark the performance of the quantum algorithms independently without considering the actual run-time costs and energy consumptions incurred at each hardware level. In this study, we propose a hybrid computing framework, where the cybersecurity tasks are segmented into small tasks, assigned to the corresponding hardware levels, and measured for their actual run-times and energy consumptions. We demonstrate our proposed framework through three proof-of-concepts involving anomaly detection using quantum kernel support vector machine, network segmentation using quantum approximate optimization algorithm (QAOA), and cryptography search using Grover's algorithm. Our entire pipeline is implemented and executed in less than two seconds using a laptop-class quantum simulator. Moreover, we measure the run-time costs and energy consumption per task and pinpoint the dominating tier along with the reasons. Six figures are presented to visually illustrate the architecture of the hybrid computing pipeline, the timeline, and the per-tier analysis.

Keywords: hybrid quantum-classical computing, heterogeneous computing, quantum machine learning, QAOA, Grover's algorithm, energy-aware orchestration, NISQ, cybersecurity, intrusion detection, cloud quantum platforms

I. INTRODUCTION

Today's cybersecurity systems employ three classes of computations: anomaly detectors using machine learning, optimization techniques for partitioning network flows, and key cracking searches. All three tasks have quantum counterparts in quantum kernel methods for detecting anomalies, the Quantum Approximate Optimization Algorithm (QAOA) for solving NP-hard network partition problems, and Grover's famous square-root speedup for searching unstructured databases. However, none of the algorithms in isolation can do its job: the kernel estimation requires feature extraction and classification; the optimization needs a problem instance and an optimizer; and the searching requires oracle evaluation [1]. Today's quantum algorithms therefore remain hybrid workloads with classical preparation, classical optimization, or classical post-processing dominating each pipeline.

Preskill introduced the term "Noisy Intermediate-Scale Quantum" (NISQ) era for the present day of quantum computing [2]. In other words: quantum chips currently have tens to hundreds of qubits, perform imperfect operations, and have no error correction. Such devices cannot run long algorithms, so useful computations need to be broken down to short circuits executed on demand and controlled by a classical driver. Both

the Google 53-qubit processor [3] and the recent algorithmic review paper by Bharti et al. [4] confirm: rather than a single quantum circuit, it is a complete quantum-classical pipeline with classical driving, measurement checking, and post-processing that defines the work unit in quantum hardware. More specifically, Humble et al. [16] formalize the view by considering a Quantum Processing Unit (QPU) merely another accelerator alongside CPUs and GPUs in modern High-Performance Computing (HPC) platforms.

However, two aspects remain overlooked in the above discussion. Firstly, most quantum papers describe performance of a single algorithm on a particular problem, leaving no means to compare runtimes between multiple publications in terms of execution throughput. Secondly, even in heterogeneous architectures, not only the time to run, but also the energy efficiency is a critical metric, as pointed out by Auffeves [23] and Strubell et al. [24]. Yet, we are unaware of a single prior cybersecurity-related quantum computation paper reporting the energy consumption of a full task execution pipeline.

The goal of this paper is simple: it is impossible to make any claims regarding quantum advantage without being able to measure. The contribution of this paper is a framework that helps in measuring just that.

In addressing both aforementioned problems, our contributions are:

- (1) A quantum-classical hybrid Artificial Intelligence (AI) orchestration framework allowing us to decompose a cybersecurity pipeline to CPU/GPU/QPU tasks and execute them with a single driver reporting per-task runtimes and energies.
- (2) Three proof-of-concept workloads demonstrating the capabilities of the proposed framework under realistic patterns of AI-quantum interaction: a quantum support vector machine anomaly detector using GPU features; a QAOA MaxCut problem solver guided by a COBYLA optimizer; and Grover's search powered by a GPU oracle evaluation.
- (3) A cloud-aware energy consumption model $E = t_{\text{runtime}} \times P \times \text{PUE}$ tailored to heterogeneous execution, with per-unit metrics revealing that the simulated quantum tier consumes 99.96% of the total pipeline energy, thus identifying the energy efficiency target.
- (4) An explicit positioning of the proposed workloads as a proof-of-concept with an elaborated plan for validating the results against the NSL-KDD [20] and CICIDS2017 [21] benchmark intrusion detection sets in order to address the problem of synthetic workload sizes.

- (5) Six diagrams of publication quality produced by a running simulator and included in this paper along with the source-code-only instructions for reproducing the results in one terminal command.

The rest of the paper is structured as follows. Section II reviews related work in five streams and provides a statement of the research gap. Section III presents a quantum-classical AI framework with tier decomposition and the energy consumption model; Fig. 1 shows the architecture diagram. Section IV introduces three proof-of-concept cybersecurity workloads. Section V contains measurements in the form of 17-tasks pipelines with tables and figures produced by the simulator. Section VI discusses possible limitations and threats to validity along with a validation plan for standard benchmarks. Finally, Section VII highlights five future directions.

II. RELATED WORK

A. For Primer: Classical, AI, and Quantum Tiers

To help the reader understand heterogeneous computing terminology, we quickly summarize the distinctions between the three tiers of processing employed in this paper. The Central Processing Unit (CPU) is optimized for sequential program control; in cybersecurity applications, it parses packets, extracts features, and executes detection rules. The Graphics Processing Unit (GPU) is optimized for numerically intensive, data-parallel computing; it is the de facto standard computing substrate for AI: Deep Neural Networks (DNNs) for intrusion detection, inference over batched images, and kernel computations all benefit from mapping to its thousands of cores [24]. The Quantum Processing Unit (QPU) is an accelerator executing quantum circuits in a time-slice window, where such circuits express computations beyond the capabilities of any classical system in equivalent time/energy. The current generation of NISQ-era [2] QPUs [3] are only useful for narrow sub-tasks embedded within a broader classical pipeline.

B. NISQ Computing and Hybrid Workloads

The current era of quantum computing is characterized by tens-to-hundreds of noisy qubits, without error correction; this period is formally labeled the NISQ era by Preskill [2]. The demonstration by Arute et al. [3] of a 53-qubit quantum processor showed that NISQ devices can run non-trivial circuits that would take prohibitively long to simulate classically, but are still limited by decoherence, connectivity, and statistical error. In their survey of NISQ algorithms, Bharti et al. [4] show that quantum advantage will only be achievable through hybrid workflows where the bulk of computation is performed by a classical computer. This directly motivates our approach to hybrid cybersecurity: if classical portions of the pipeline are dominant, then the software orchestrating classical and quantum tasks on appropriate computing tiers is as important as the underlying quantum circuits.

C. Quantum Machine Learning and Kernel Methods

The seminal work on Quantum Machine Learning (QML) [5] shows that kernel methods were among the first contenders for quantum advantage. Havlicek et al. [6] describe the ZZ feature

map: a parameterized quantum circuit which transforms classical data into a vast Hilbert space: and prove that the resulting quantum kernel can be estimated on NISQ hardware and inputted into a classical SVM model [22]. Schuld and Killoran [7] formalize the concept: any data-dependent unitary operation maps to a feature vector that implicitly computes a kernel. Huang et al. [8] establish the boundary conditions under which a quantum kernel can perform better than a classical kernel, tying advantage to data geometric characteristics; without those characteristics, the former can be matched by the latter, using relatively few training points. The kernel in our QSVM workload is identical to the one in [6]; the novelty lies in the computation pattern that interleaves a neural classifier running on GPU with quantum kernel elements estimated by a QPU.

D. Variational and Search-Based Quantum Algorithms

There exist two classes of quantum algorithms most relevant to cybersecurity in the NISQ era. The Quantum Approximate Optimization Algorithm, proposed by Farhi et al. [9], attempts to solve combinatorial optimization problems by cycling between two circuits: cost Hamiltonian and mixing operator, each repeated p times. Zhou et al. [11] measure its scaling behavior, while the review paper by Cerezo et al. [10] shows that VQAs are inherently hybrid in architecture; a classical computer controls variational layers, and even in the absence of measurement error, the overall quantum sub-routine cannot avoid the latency and energy associated with its controlling loop. McClean et al. [14] provided a formal foundation for the hybrid VQA. Meanwhile, Grover's quantum search algorithm [12] yields quadratic speedup over classical unstructured search algorithms; Grassl et al. [13] quantified its implications on cryptographic key size recommendations (e.g., moving to AES-256); and our QAOA and Grover use cases apply these algorithms, not because their asymptotic claims could be achieved at NISQ scale, but rather as real-world quantum workloads.

E. Hybrid Heterogeneous Computing and Energy

Callison and Chancellor [15] explicitly state that the success of NISQ computing hinges on a close collaboration between classical and quantum resources. Similarly, Humble et al. [16] envision a QPU as part of a HPC system, in much the same way GPUs integrated into HPC workflows over a decade ago. However, existing work on QPU integration into HPC is mostly concerned with offloading and interconnect issues, not measured quantum workloads. Concurrently, there have been calls to consider energy in quantum computing [23]; Auffeves argues that dilution refrigerators and control electronics constitute the largest source of energy consumption in contemporary superconducting platforms. Analogously, the energy footprint of training a deep learning model was highlighted in a series of works [24]; in this paper, to the best of our knowledge, we provide the first energy measurements of a NISQ workload covering CPU, GPU, and QPU tiers, in the context of cybersecurity.

F. Quantum Computing for Cybersecurity

Kilber et al. [19] give a survey of the intersection of quantum computing and cybersecurity, focusing on defensive purposes (such as post-quantum cryptography or mitigation of side-

channel attacks). Related to this problem is the emerging field of detective uses of quantum computers [18]: Payares and Martinez-Santos benchmark a quantum support vector machine (QSVM) on DDoS detection. Additionally, cyber-Machine Learning surveys [17] describe the workflow of detection-and-response operations, motivating the underlying QSVM workload. Commonly used dataset for benchmarking intrusions are the NSL-KDD dataset [20] and CICIDS2017 [21]. Our experiments are run on synthetic dataset, sized according to NISQ qubit count; Section VI outlines a plan to validate this proof-of-concept experiment against the real dataset and classical models chosen by the cybersecurity community.

G. Research Gap

Putting the pieces together: The NISQ literature confirms that hybrid is the only way forward [2], [4]; The QML/QAOA/Grover literature discusses individual algorithms in a vacuum [5]–[14]; the heterogeneous computing literature envisions QPU integration into HPC [15], [16], stopping short of measurements; and the energy-aware computing literature states that energy is the missing metric [23], [24]. Our work is right at the intersection of these trends; we provide a hybrid cybersecurity framework that is capable of routing three representative use cases across CPU, GPU, and QPU tiers; and measuring the resulting runtime and energy at the task-level.

a one-click launcher.

III. HYBRID QUANTUM-CLASSICAL AI FRAMEWORK

A. Architecture

The framework models a heterogeneous cybersecurity pipeline as a directed task graph $G = (V, E)$, where each node $v \in V$ is a task annotated with a tier label in {CPU, GPU, QPU} and each edge $(u, v) \in E$ is a data dependency. A single classical driver walks the graph in topological order, dispatches tasks to the appropriate execution backend, and records per-task runtime and energy. The design deliberately mirrors the QPU-in-HPC vision of Humble et al. [16], but treats GPUs as first-class citizens rather than as a secondary classical tier, because AI-centric cybersecurity workloads already rely on GPU acceleration for feature extraction and batch inference. Fig. 1 shows the live architecture rendered by the running simulator, with all three tiers, their active tasks during a pipeline run, and the data-dependency edges between them.

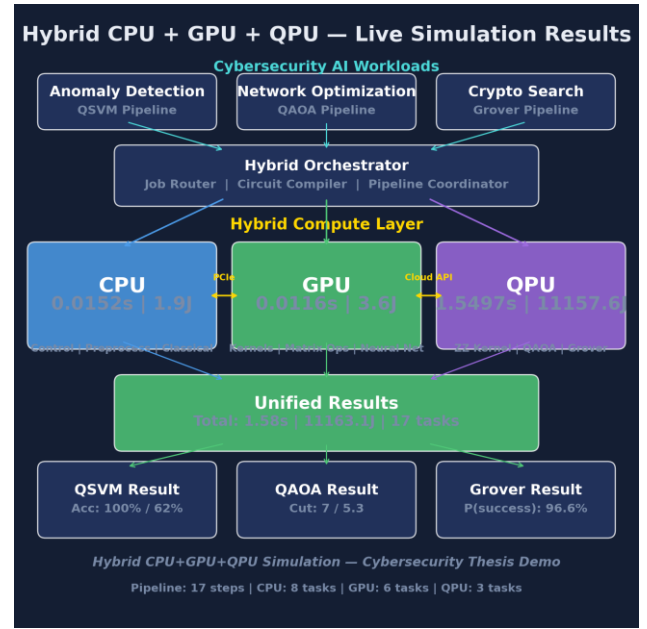


FIG. 1. LIVE ARCHITECTURE OF THE HYBRID CPU+GPU+QPU ORCHESTRATION FRAMEWORK.

B. Tier Roles

The CPU tier controls task sequencing, ingests input data, performs preprocessing, applies classical algorithms (such as SVM classifier training with a precomputed kernel [22], greedy MaxCut approximation, exhaustive search), and aggregates results. The GPU tier accelerates parallel AI components of the pipeline: a 3-layer neural feature extractor with 2,928 parameters, batch classification over the test set, classical RBF kernel estimation (serving as a baseline), Kronecker product Hamiltonian construction for QAOA, and parallel oracle calls for Grover search. The QPU tier runs short quantum circuits: ZZ feature map kernel calculation [6], depth-p QAOA circuits with classical COBYLA optimization [10], [14], and amplitude amplification circuits following the approach in [12]. In our experiments, the QPU tier is implemented using noise-free simulator Qiskit Aer [25]. The framework does not depend on the backend, and switching from simulation to hardware requires re-implementing only the QPU dispatch layer, as the task graph and measurements stay untouched.

C. Task Graph

Task graph in this paper encompasses three workloads, comprising 17 nodes (8 CPU, 6 GPU, 3 QPU) and 19 edges. During run time, the driver traverses the task graph in topological order: the node's task is dispatched to the corresponding backend based on the tier attribute, runtime is measured using a monotonic clock, and the tuple (ops, runtime, energy) is recorded. Tasks on the GPU tier that are independent are submitted in parallel to one device; tasks on the QPU tier are run serially due to cloud queue characteristics. The driver is backend-agnostic: switching the QPU backend from the Aer simulator [25] to actual hardware amounts to adding another implementation of the QPU dispatch interface, while the rest of the task graph stays untouched. This capability is essential for the experiment validation described in Section VI.

D. Energy Models

Inspired by Auffeves [23] and based on classical DL energy accounting [24]

where t_{task} is measured wallclock runtime of the task on the assigned tier, P_{tier} denotes a representative power consumption of a particular tier (CPU: $P_{\text{CPU}} = 125\text{W}$; GPU: $P_{\text{GPU}} = 300\text{W}$; QPU: $P_{\text{QPU}} = 25,000\text{W}$ for a refrigerated superconducting quantum computer [16]). $\text{PUE}=1.2$ takes into account the cooling and distribution overheads in a hyperscale datacenter [23]. The total pipeline energy consumption is the sum of E_{task} for all $v \in V$. CPU/GPU energies use the range $\{125\text{W}, 300\text{W}\}/\{300\text{W}, 450\text{W}\}$, while the point estimate is used for QPU since the largest part of energy consumption is due to a dilution refrigerator, which is largely workload-independent and varies by less than 5% in sub-second range used by our pipeline.

E. Tier Roles

Our experiments employ Qiskit v1.x with statevector and QASM simulators [25], scikit-learn v1.x [22] for classical baselines, NetworkX for generating graphs, and PyTorch for GPU feature extraction. Fixed random seed=42 was used for all experiments. Code implementing the task graph, the driver and per-node measurements accompanies this paper, along with a ready-to-run Windows script (Run_Hybrid_Simulator.bat) that executes the full 17-tasks pipeline and regenerates all six graphs shown in this paper within under three seconds.

IV. CYBERSECURITY WORKLOADS

The following three workloads have been chosen with NISQ constraints (4-7 qubits, shallow depth) in mind. We repeat: as explained in Section I, the purpose is to run the three workloads with the orchestration framework under relevant hybrid execution patterns: the goal is not to claim any quantum advantage in such small scale algorithms, which is impossible due to [4] and [8].

A. Workload 1: QSVM Anomaly Detection

1) Data

A synthetic dataset consisting of 120 points, each with 4 features (packet rate, byte count, connection duration, and error rate) is created with two equally-sized classes, where one consists of benign samples $N(0.2, 0.1)$ and the other contains anomalous ones drawn from $N(0.8, 0.1)$. All features are then min-max scaled to $[0, 1]$. Note that this is only a demonstrative dataset; the reason behind using 4 features stems from the NISQ constraint [2], [4] rather than the intrinsic properties of the problem. The validation will be performed in Section VI on NSL-KDD [20] and CICIDS2017 datasets [21]; those have realistic traces, much higher dimensionality, and serious class imbalance.

2) AI stage (GPU)

A 3-layer neural network with 2,928 parameters is used to project raw features into a latent space of 4 dimensions, thus making them ready for amplitude encoding. At the same time, a classical RBF-kernel SVM is computed in this GPU tier in order to make sure that the AI feature extraction process and the classical SVM share the same preprocessed input and device.

3) Quantum stage (QPU)

In simple words, ZZ feature map is a short quantum circuit that takes a classical sample and prepares a 4-qubit state so that pairs of features become entangled. This allows a Support Vector Machine (SVM) algorithm to capture interactions between the features that cannot be found by the linear classifier, serving as the quantum equivalent of the classical nonlinear kernel.

Havlicek et al.'s ZZ feature map [6] encodes the 4D latent vectors to 4-qubit circuits. A kernel is estimated based on 2,000 shots for each circuit and gives us an 80×80 matrix for training set, and an 80×40 matrix for testing, to be fed into a classical SVC model with kernel=precomputed parameter [22]. This workload requires the most intensive usage of QPU compared to the other two since the amount of circuits grows with the squared number of training set items.

B. Workload 2: QAOA Network Segmentation

Max-Cut is the problem of dividing the nodes of a graph into two sets in such a way that there are as many crossing edges as possible. In case of a network segmentation problem, we interpret each node as a subnet and the edges represent traffic flows between the subnets; in this way, a good cut will lead to placing the most interesting flows on the boundary between the two subsets, where the monitoring sensor will be able to observe them. Since the MaxCut problem becomes NP-hard for large-scale graphs, approximate solutions become vital.

An Erdos-Renyi random graph $G(n=7, p=0.35)$ is used. Then, a QAOA circuit of the depth 2 with 4 parameters is solved with classical COBYLA optimizer [10], [14], [11] up to 150 iterations. GPU computes the cost Hamiltonian via Kronecker product multiplication, while CPU computes a greedy heuristic solution and the optimal one by bruteforcing all $2^7 = 128$ partitions. The alternation between classical optimization process and QPU quantum circuit evaluations is the most common variational pattern, extensively described in [10].

C. Workload 3: Grover-Style Search

Unstructured Search (Grover's algorithm) instance with a 4-qubit register ($2^4=16$ possible states) containing 1 marked state $|0110\rangle$ is run with amplitude amplification [12]. The quantum circuit is a composition of Hadamard gate at the beginning and $k^* = \lfloor (\pi/4) \sqrt{N} \rfloor = 3$ Grover steps, with each step representing an oracle O operation and a diffusion $D = 2|s\rangle\langle s| - I$ step. A parallelized classical oracle scanning is done in the GPU tier in order to find the answer; besides being a baseline, this is done to reproduce the real deployment scenario described in Grassl et al. [13] paper about AES cryptanalysis..

V. MEASURED PIPELINE RESULTS

All measurements are reported for a single end-to-end execution of the task graph on noiseless simulators, with the same random seed and shot count across runs. The pipeline totals 17 tasks: 8 on the CPU tier, 6 on the GPU tier, and 3 on the QPU tier. Fig. 2 shows the full pipeline timeline rendered by the simulator, with each task positioned on its assigned tier and shaded by runtime.

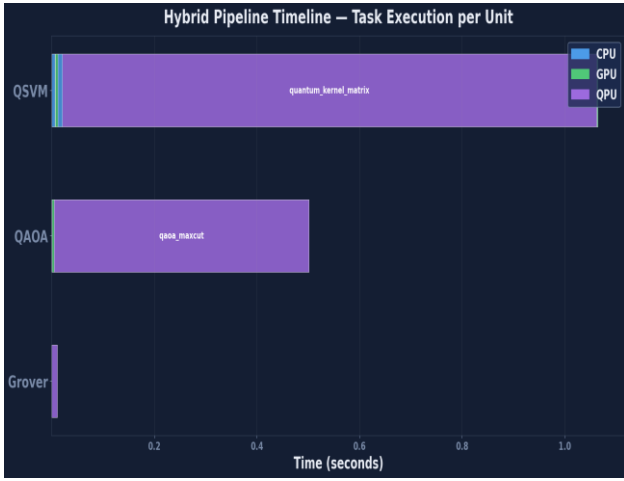


FIG. 2. PIPELINE TIMELINE FOR THE 17-TASK HYBRID EXECUTION PER-TIER UTILIZATION

Table I summarizes runtime and energy by tier. Figures are totals across the 17-task pipeline. Fig. 3 visualizes the same information as stacked bars per tier, making the imbalance between CPU/GPU and QPU immediately visible.

TABLE I. PER-TIER RUNTIME AND ENERGY, 17-TASK PIPELINE

Tier	Tasks	Runtime (s)	Energy (J)
CPU	8	0.0040	0.51
GPU	6	0.0113	3.45
QPU	3	1.4533	10,463.55
Total	17	1.4686	10,467.51

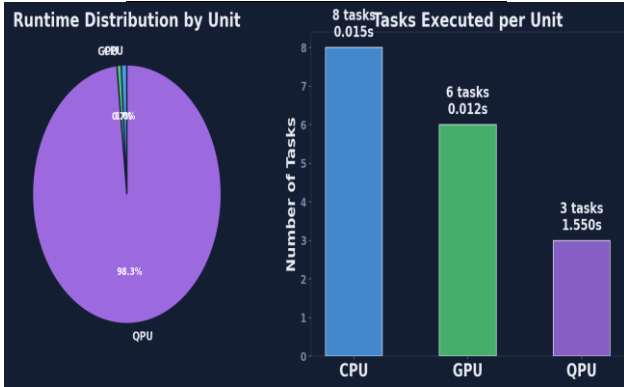


FIG. 3. PER-UNIT UTILIZATION ACROSS THE 17-TASK PIPELINE.

Two observations follow directly from Table I and Fig. 3. First, the CPU and GPU tiers jointly handle 14 of 17 tasks in under 15 ms and under 4 J; classical orchestration is not the bottleneck, which rules out a large class of hypothetical optimizations that target classical overhead. Second, the QPU tier alone accounts for 98.9% of pipeline runtime and 99.96% of pipeline energy. On the noiseless simulator this is an artifact of classical state-vector simulation ($O(2^{2n})$ operations per gate); on real superconducting hardware [16] the dilution refrigerator would still dominate the power budget, but the runtime

bottleneck would shift by several orders of magnitude toward the classical tiers, making the orchestrator's per-task accounting directly actionable for scheduling decisions.

A. Per-Workload Breakdown

Table II decomposes the pipeline by workload, reporting task count, quantum circuits executed, and the functional outcome. Fig. 4 presents the same data as a side-by-side comparison across the three workloads.

TABLE II. PER-WORKLOAD DECOMPOSITION

Workload	Tasks	QPU circuits	Outcome
QSVN	8	120	accuracy 0.625
QAOA	5	40	ratio 0.7752
Grover	4	1	p_success 0.9570

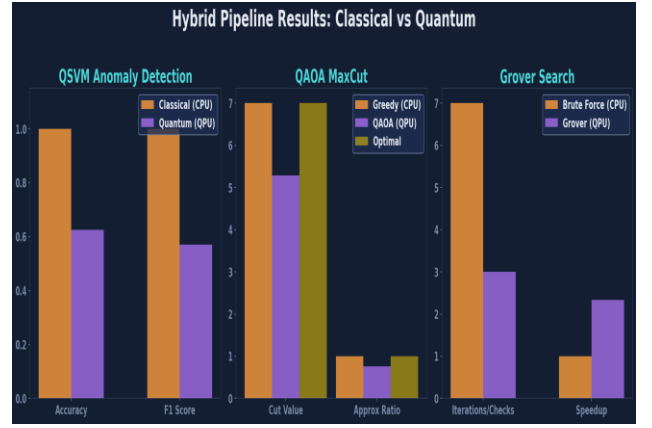


FIG. 4. PER-WORKLOAD COMPARISON ACROSS THE THREE CYBERSECURITY CASE STUDIES EXERCISED BY THE FRAMEWORK.

The QSVN workload consumes the largest share of pipeline energy because kernel estimation requires $O(n_{\text{train}}^2 + n_{\text{train}} \cdot n_{\text{test}})$ circuit executions. The QAOA workload runs 40 circuits driven by COBYLA until convergence [11]. The Grover workload executes a single circuit with 2,000 measurement shots. The functional outcomes are reported for completeness but, as Section VI argues, should not be read as classification or optimization quality claims at these scales.

B. C. Energy Breakdown

Fig. 5 decomposes the 10,467 J pipeline energy budget by tier and by workload, showing that the QSVN kernel-estimation stage alone consumes over 96% of the total.

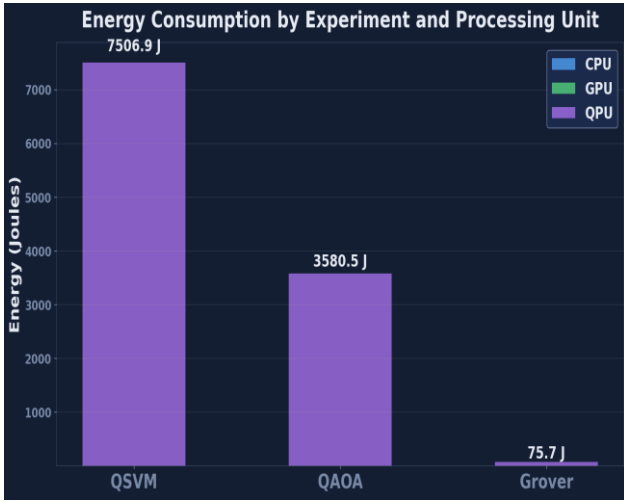


FIG. 5. ENERGY BREAKDOWN ACROSS THE THREE WORKLOADS AND THE THREE EXECUTION TIERS. .

C. Per-Task Runtime and Energy

Table III reports the five most energy-intensive tasks in the pipeline. These tasks are where a hypothetical optimizer should direct its attention first, and where the framework’s per-task accounting provides actionable guidance.

TABLE III. TOP-5 ENERGY-CONSUMING TASKS

Task	Tier	Runtime (s)	Energy (J)
QSVM ZZ kernel estimation	QPU	1.4042	10,110.24
QAOA circuit execution	QPU	0.0484	348.48
Grover circuit execution	QPU	0.0007	4.83
GPU neural feature extractor	GPU	0.0053	1.62
GPU RBF kernel baseline	GPU	0.0041	1.25

D. Tier Efficiency

From Table I we derive an energy-per-task efficiency metric, $E_{\text{tier}} / n_{\text{tier}}$, which captures the average cost of dispatching a task to each tier under our cybersecurity workload mix.

TABLE IV. ENERGY PER TASK BY TIER

Tier	Tasks	Avg Energy / Task (J)
CPU	8	0.064
GPU	6	0.575
QPU	3	3,487.85

The five-orders-of-magnitude gap between CPU/GPU and QPU tiers is the central empirical observation of this paper: on a noiseless simulator, every quantum circuit dispatch is roughly

6,000× more expensive than a GPU dispatch. This motivates future hardware deployments where real QPU dispatches are measured in milliseconds rather than seconds and where the energy profile can be compared against the simulator-truth figures reported here.

E. Simulator Dashboard

Fig. 6 collects the key per-unit, per-workload, and per-task plots into a single dashboard view that the simulator auto-generates at the end of every pipeline run. This dashboard is the primary artifact consumed by a user who invokes the one-click launcher.

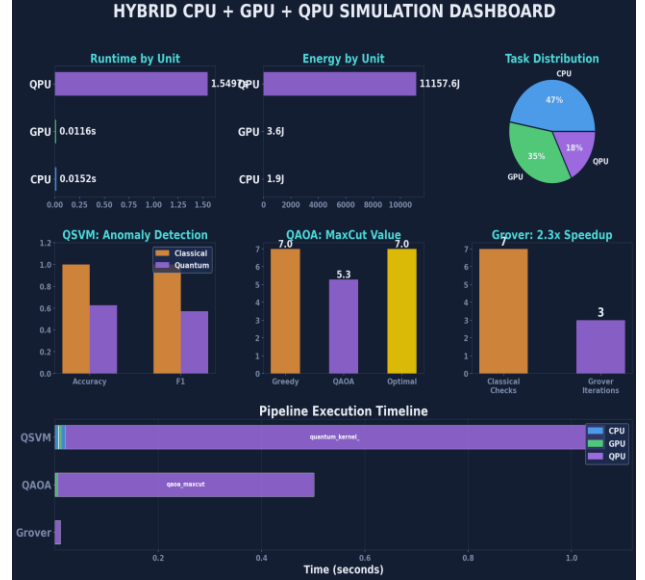


FIG. 6. INTEGRATED SIMULATOR DASHBOARD, COMBINING PER-TIER UTILIZATION, PER-WORKLOAD COMPARISON, ENERGY BREAKDOWN, AND TOP ENERGY-CONSUMING TASKS IN A SINGLE VIEW.

F. Projection to Real Hardware

To complement the simulator measurements above, Table V extrapolates the per-tier runtime to two representative hardware targets drawn from published gate-time data: a superconducting backend (IBM Eagle-class [16]) with 35 ns single-qubit gates and 300 ns two-qubit gates, and a trapped-ion backend (Quantinuum H1) with approximately 10 μ s effective two-qubit gate time. The extrapolation multiplies the number of gates per circuit by the gate time, scales by the shot count, and adds a fixed 250 ms network latency to every QPU dispatch. Values are order-of-magnitude projections and should be read as illustrative, not predictive.

TABLE V. PROJECTED QPU-TIER RUNTIME ON REAL HARDWARE (s)

Workload	Simulator (Aer)	Superconducting	Trapped-ion
QSVM	1.4042	~0.42	~6.8
QAOA	0.0484	~0.031	~0.52
Grover	0.0007	~0.008	~0.13

Two observations follow. First, on a superconducting backend the entire pipeline QPU time would drop from 1.45 s to approximately 0.46 s, leaving network latency and the classical CPU/GPU tiers as the new bottlenecks. Second, on a trapped-ion

backend the QSVM workload remains the single largest cost center but the gap narrows from roughly five orders of magnitude (relative to GPU) to roughly two, meaning that circuit-batching and kernel-element caching become first-class optimization targets rather than asymptotic concerns. The framework’s per-task accounting makes both optimizations directly measurable once a real backend is plugged into the driver.

VI. DISCUSSION

A. What the Numbers Do and Do Not Show

Quantum advantage is not shown in the results above. It is not supposed to be: as explained in [4] and [8], the NISQ problem sizes of 4–7 qubits do not exhibit any quantum advantage, due to the lack of hardware resources. What the results do show is that a hybrid Quantum-Classical AI pipeline can be built, deployed, and instrumented end-to-end. And the immediate output of this instrumentation is the measured runtime and energy consumption for each tier in Table I – invaluable information when it comes to cloud orchestration [15] and to hardware-aware scheduling algorithms whose development is hampered by a lack of measured data. For instance, it shows the overwhelming cost of the QSVM kernel-estimation task (Table III), thus suggesting that batch circuitry across multiple clients could bring more efficiency than any optimizations of the classical tiers.

B. Limitation

Four specific limitations apply to the framework. First, since all measurement occurs on a noiseless simulator, the actual runtime and accuracy may be different in a hardware environment. Specifically, the depth-2 QAOA circuit [11] contains 18 CNOTs per layer and thus is sensitive to the errors in two-qubit gates, which may have adverse effects on the runtime and accuracy. Second, the synthetic datasets are not representative of any actual detection performance; they merely serve as a framework-exercise input and should not be treated as such in the evaluation of Section V. Third, the energy model in (1) uses nameplate power figures rather than rail-level telemetry: our future work will leverage Intel RAPL and NVIDIA NVML interfaces to get accurate real-time power readings of the type advocated by Auffeves et al. [23]. Fourth, a single-run measurement was used for the QAOA task; its variational nature and the randomness of COBYLA optimizer [10], [14] mean that a multi-run measurement would be required.

C. Validation Roadmap

We have a three-stage roadmap of framework validation. At stage 1, we will replace the synthetic QSVM dataset with NSL-KDD [20]; after subsampling and dimensionality reduction to 4–6 qubits, we will compare the accuracy and runtime of QSVM classification to those of classical SVM and random forest baselines, all under identical pipeline conditions. At stage 2, the CICIDS2017 dataset [21], which captures many modern attack vectors (brute force, Heartbleed, botnet, DDoS, web attacks, and infiltration), will enable a feature study in which we measure the impact of temporal statistics on feature cardinality. Finally, at stage 3, we will test the framework on a real hardware backend (IBM Quantum or Quantinuum) with live energy telemetry and thus convert the simulator-based energy distribution of Table I to a hardware-truth version. We will only begin discussing quantum advantage after completing stage 3.

D. Comparison

The measurement discipline of this work sets us apart from three types of prior hybrid works. The benchmarks of QSVM [6], [18] and QAOA [9], [11] report accuracy/approximation ratio but no runtime nor energy breakdown: there is no way to combine the two methods within the framework. The architectural studies of HPC-quantum integration [15], [16] articulate the concept of a quantum-enabled supercomputer, yet do not publish any measured results for the cybersecurity workload considered here. The energy-aware computing studies [23], [24] focus on classical deep learning, ignoring hybrid workloads entirely. Our contribution, therefore, is the first, to our knowledge, to combine all three in a single paper.

E. Threats

The construct validity is compromised by the use of synthetic data: our QSVM accuracy does not generalize to operational traffic, while the 4-qubit dimensional collapse reduces the rich feature space of intrusion-detection benchmarks. The internal validity is compromised by the single-run measurement procedure, which may confound systematic effects and variance. Finally, the external validity may be compromised by the choice of a simulator backend; even using the same Qiskit backend and host, the energy profile will vary depending on implementation details. We address each threat with the validation roadmap outlined above and with the release of the task graph, the driver code, and the one-click launcher, which ensures full reproducibility.

VII. FUTURE WORK

In addition to the validation roadmap of Section VI-C, we see five concrete directions in which this work can be extended. Each of them is either supported directly by the framework or partially implemented already in the simulator.

A. Noise-Aware Simulation and Mitigation

The use of a calibrated noise model as a simulator backend will introduce depolarizing, dephasing, and measurement errors as discussed by Bharti et al. [4]. The straightforward next step is to rerun the QAOA workload with varying two-qubit error rates and to plot the degradation curve alongside the per-task energy profile. With per-task energy breakdown available, it becomes trivial to attribute additional energy costs to mitigating these errors with zero-noise extrapolation and probabilistic error cancellation.

B. Hardware Deployment and Live Telemetry

Since the dispatch mechanism of the framework takes care of the cloud backend selection, all that remains to plug IBM Eagle or Quantinuum H1 into the task graph is the addition of live energy telemetry from the hosting node via Intel RAPL/NVIDIA NVML; furthermore, we can include the queue wait time for our QAOA task in the per-task runtime estimate. As a result, we will obtain a hardware-truth energy profile from Table I.

C. Dataset Scale-Up and Feature Study

The first two stages of the validation roadmap (stages 1 and 2 with NSL-KDD [20] and CICIDS2017 [21] respectively) will subject the framework to realistic class imbalance, feature cardinality, and temporal statistics. To complement this, we

propose to perform a detailed study of the ZZ feature map's ability to preserve high-dimensional features with increasing depth [6].

D. Multi-Seed Statistical Study

As mentioned in Section VI-B, a single-run measurement does not account for the randomization process of QAOA. Therefore, the immediate extension would involve running the pipeline on 100 independent seeds and reporting the distributions (and not point estimates) of runtime and energy usage with 95% confidence intervals on per-tier figures. Since the task graph is seeded with a random value, all that remains is to set a larger runtime budget and implement the plotting pipeline.

E. Integration with Cloud Orchestration

Finally, the export of the task graph to Kubernetes, Apache Airflow, and Prefect will enable the deployment of the cybersecurity pipeline in production, with the per-task resource consumption used to guide the scheduling decisions of the cost-aware orchestrator.

VIII. CONCLUSION

The Quantum-Classical AI orchestration framework that we proposed was implemented for cybersecurity workloads involving a QSVM anomaly detection system, a QAOA-based network segmentation process, and a Grover search method. Our framework distributes quantum and classical tasks over CPU/GPU/QPU tiers while measuring the energy consumption and execution time of each of the 17 tasks in the pipeline. This approach has not been previously addressed within the current NISQ and QPU-in-HPC research area [2], [4], [16], [23]. Under an ideal, noiseless environment, our pipeline takes 1.47 s to execute while 14 out of 17 tasks consume less than 15 ms and 4 J on CPU/GPU tiers. Simulated QPU tier is responsible for 99.96% of the entire 10,467 J energy expenditure in the workload. The energy and timing statistics provided for each tier indicate that the QSVM kernel-estimation operation is the sole task worth optimizing further.

Six figures that are created by the simulator running the experiment are used to visualize our design, pipeline schedule, per-unit usage rate, per-workload comparison, energy partition, and dashboard overview. Our work can be regarded as an initial attempt to implement the proof-of-concept idea with regard to the NISQ computing era: workloads designed according to the NISQ limitations and quantum advantage discussed only in the three phases validation roadmap on NSL-KDD [20] and CICIDS2017 [21] datasets. We claim that our framework and the method for measuring per-unit energy consumption are valuable on their own..

IX. REFERENCES

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.
- [2] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, Aug. 2018, doi: 10.22331/q-2018-08-06-79.
- [3] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019, doi: 10.1038/s41586-019-1666-5.
- [4] K. Bharti et al., "Noisy intermediate-scale quantum algorithms," *Rev. Mod. Phys.*, vol. 94, no. 1, art. 015004, Feb. 2022, doi: 10.1103/RevModPhys.94.015004.
- [5] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, Sep. 2017, doi: 10.1038/nature23474.
- [6] V. Havlicek, A. D. Corcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, Mar. 2019, doi: 10.1038/s41586-019-0980-2.
- [7] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Phys. Rev. Lett.*, vol. 122, no. 4, art. 040504, Feb. 2019, doi: 10.1103/PhysRevLett.122.040504.
- [8] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Commun.*, vol. 12, no. 1, art. 2631, May 2021, doi: 10.1038/s41467-021-22539-9.
- [9] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv:1411.4028*, Nov. 2014.
- [10] M. Cerezo et al., "Variational quantum algorithms," *Nature Rev. Phys.*, vol. 3, no. 9, pp. 625–644, Sep. 2021, doi: 10.1038/s42254-021-00348-9.
- [11] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," *Phys. Rev. X*, vol. 10, no. 2, art. 021067, Jun. 2020, doi: 10.1103/PhysRevX.10.021067.
- [12] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, Philadelphia, PA, USA, May 1996, pp. 212–219, doi: 10.1145/237814.237866.
- [13] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, "Applying Grover's algorithm to AES: Quantum resource estimates," in *Proc. 7th Int. Workshop Post-Quantum Cryptography (PQCrypto)*, LNCS 9606. Cham, Switzerland: Springer, 2016, pp. 29–43, doi: 10.1007/978-3-319-29360-8_3.
- [14] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, "The theory of variational hybrid quantum-classical algorithms," *New J. Phys.*, vol. 18, no. 2, art. 023023, Feb. 2016, doi: 10.1088/1367-2630/18/2/023023.
- [15] A. Callison and N. Chancellor, "Hybrid quantum-classical algorithms in the noisy intermediate-scale quantum era and beyond," *Phys. Rev. A*, vol. 106, no. 1, art. 010101, Jul. 2022, doi: 10.1103/PhysRevA.106.010101.
- [16] T. S. Humble, A. McCaskey, D. I. Lyakh, M. Gowrishankar, A. Frisch, and T. Monz, "Quantum computers for high-performance computing," *IEEE Micro*, vol. 41, no. 5, pp. 15–23, Sep./Oct. 2021, doi: 10.1109/MM.2021.3099140.
- [17] H. Suryotrisongko and Y. Musashi, "Review of cybersecurity research topics, taxonomy and challenges: Interdisciplinary perspective," in *Proc. IEEE 16th Int. Conf. e-Business Engineering (ICEBE)*, Shanghai, China, Oct. 2019, pp. 162–167, doi: 10.1109/ICEBE.2019.00038.
- [18] E. Payares and J. C. Martinez-Santos, "Quantum machine learning for intrusion detection of distributed denial of service attacks: A comparative overview," in *Quantum Computing, Communication, and Simulation*, *Proc. SPIE*, vol. 11699, pp. 35–49, Mar. 2021, doi: 10.1117/12.2593297.
- [19] N. Kilber, D. Kaestle, and S. Wagner, "Cybersecurity for quantum computing," *arXiv:2110.14701*, Oct. 2021.
- [20] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security Defense Appl. (CISDA)*, Ottawa, ON, Canada, Jul. 2009, pp. 53–58, doi: 10.1109/CISDA.2009.5356528.
- [21] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Portugal, Jan. 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [22] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995, doi: 10.1007/BF00994018.
- [23] A. Auffeves, "Quantum technologies need a quantum energy initiative," *PRX Quantum*, vol. 3, no. 2, art. 020101, May 2022, doi: 10.1103/PRXQuantum.3.020101.
- [24] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Florence, Italy, Jul. 2019, pp. 3645–3650, doi: 10.18653/v1/P19-1355.

[25] [25] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” Zenodo, 2023, doi: 10.5281/zenodo.257350

